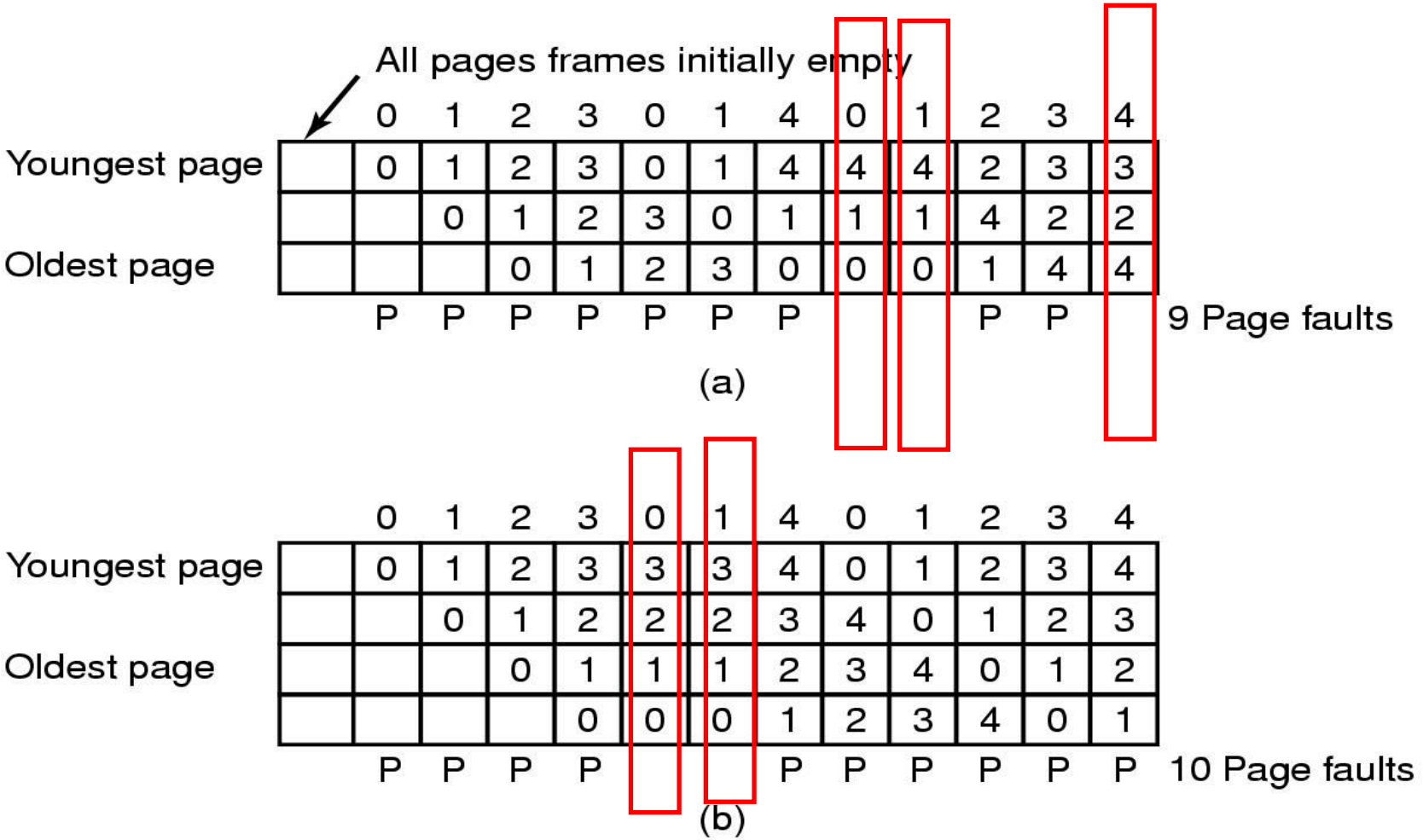
Chapter 3

Memory Management

More Pages -> Less Faults ??!!

- Using FIFO
- 0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4



More Pages -> Less Faults ??!!

- Stack property
  - An algorithm has the stack property if, for any given sequence of page references, the set of pages in memory with n frames is always a subset of the pages in memory with n+1 frames.
- It holds for the replacement algorithms which does not have the following property:
  - Let M(m,r) = set of pages present in physical memory where physical memory has m no of frames and the process is referencing the rth page

$$M(m,r) \subseteq M(m+1,r)$$

  - Ensures that **belady's anomaly** will not occur. (FIFO does not hold this, LRU does)
  - Algorithms following this property is known as Stack Algorithms

More Pages -> Less Faults ??!!

- This happens because FIFO blindly removes the oldest page, even if it will be used soon.
- This is known as belady's Anomaly
- For FIFO: Eviction based on load time, not usage. More frames change eviction order unpredictably → can lose pages that fewer frames would keep.
- FOR LRU: Eviction based on recency. More frames = strictly more history retained → smaller memory state is always contained in larger one.

- How about using LRU - do it using LRU (3 page frame)
- 0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4 → 10 page fault

Step	Ref	Memory	Fault?	Action
1	0	[0]	✅ Yes	Load 0
2	1	[0, 1]	✅ Yes	Load 1
3	2	[0, 1, 2]	✅ Yes	Load 2
4	3	[1, 2, 3]	✅ Yes	Evict 0 (oldest), load 3
5	0	[2, 3, 0]	✅ Yes	Evict 1, load 0
6	1	[3, 0, 1]	✅ Yes	Evict 2, load 1
7	4	[0, 1, 4]	✅ Yes	Evict 3, load 4
8	0	[1, 4, 0]	❌ No	Hit (0 present) → move to MRU
9	1	[4, 0, 1]	❌ No	Hit → move to MRU
10	2	[0, 1, 2]	✅ Yes	Evict 4, load 2
11	3	[1, 2, 3]	✅ Yes	Evict 0, load 3
12	4	[2, 3, 4]	✅ Yes	Evict 1, load 4

- How about using LRU - do it using LRU (4 page frame)
- 0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4 → 8 page fault

Ref #	Page	Mem	Fault?	Action
1	0	[0]	✅	Load
2	1	[0,1]	✅	Load
3	2	[0,1,2]	✅	Load
4	3	[0,1,2,3]	✅	Load
5	0	[1,2,3,0]	❌	Hit → move 0 to MRU
6	1	[2,3,0,1]	❌	Hit
7	4	[3,0,1,4]	✅	Evict 2 (LRU), load 4
8	0	[3,1,4,0]	❌	Hit
9	1	[3,4,0,1]	❌	Hit
10	2	[4,0,1,2]	✅	Evict 3, load 2
11	3	[0,1,2,3]	✅	Evict 4, load 3
12	4	[1,2,3,4]	✅	Evict 0, load 4

The Working Set Page Replacement Algorithm

- Most processes exhibit a *locality of reference*
  - during any phase of execution, the process references only a relatively small fraction of its pages.
- The set of pages that a process is currently using is called working set
  - If entire working set is in memory, no page faults!!!
- Goal: keep most of working set in memory to minimize the number of page faults suffered by a process

The Working Set Page Replacement Algorithm

- At any instant of time t, there exists a set consisting of all the pages used by the k *most recent* memory references.
- This set, w(k,t), is the working set.
- Working set may change over time

The Working Set Page Replacement Algorithm

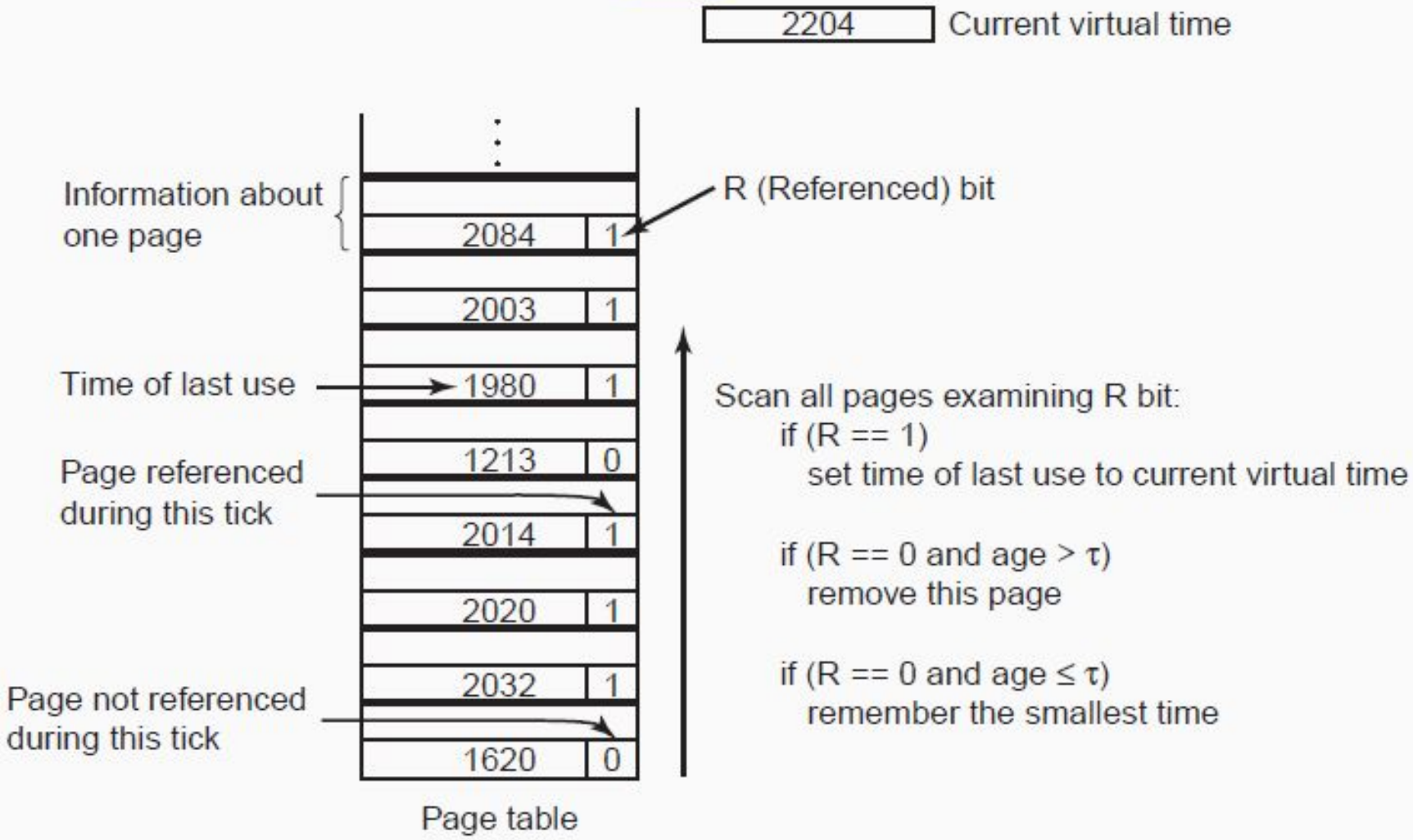
- Using a shift register of length k.
  - Very expensive
- Solution: Approximation
- Instead of using no of memory references, we may use execution time
- Idea:
  - Look back over the last T msec of time
  - Which pages were referenced?
  - This is the working set.
- Only care about how much CPU time this process has seen.
- The amount of a CPU time a process has actually used since it started is often called its **Current Virtual Time**.

The Working Set Page Replacement Algorithm

How It Works

1. Define a time window Δ (e.g., last 10,000 CPU instructions or 100 ms).
2. For each page in memory, track when it was last referenced (using hardware "reference bits" or timestamps).
3. At replacement time:
   - Scan memory for a page whose last use was outside the current window (i.e., older than `current_time - Δ`).
   - If found → evict it.
   - If all pages are in the working set → the process needs more frames (may cause thrashing).

The Working Set Page Replacement Algorithm



$$age = Current\ virtual\ time - Time\ of\ last\ use$$

The Working Set Page Replacement Algorithm

- cumbersome
  - since the entire page table has to be scanned at each page fault until a suitable candidate is located.
- An improved algorithm, that is based on the clock algorithm but also uses the working set information, is called WSClock

The WSClock Page Replacement Algorithm

- **All pages are kept in a circular list.**
- **As pages are added, they go into the ring.**
- **The "clock hand" advances around the ring.**
- **Each entry contains "time of last use" as well as R and M bit**
- **At each clock tick,**
  - the current virtual time of the process is stored in the entry of each page where the R bit is 1 and the R bit is cleared

The WSClock Page Replacement Algorithm

Since tracking exact timestamps is expensive, real systems often use an approximation:

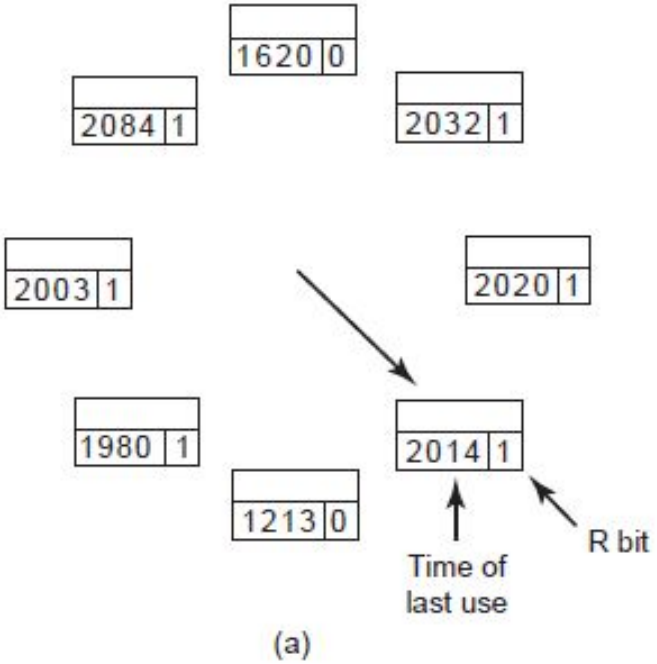
- Use a reference bit (set by hardware on access).
- Periodically clear all reference bits (simulating time windows).
- When a page's reference bit is 0, it likely hasn't been used in the current window → candidate for eviction.

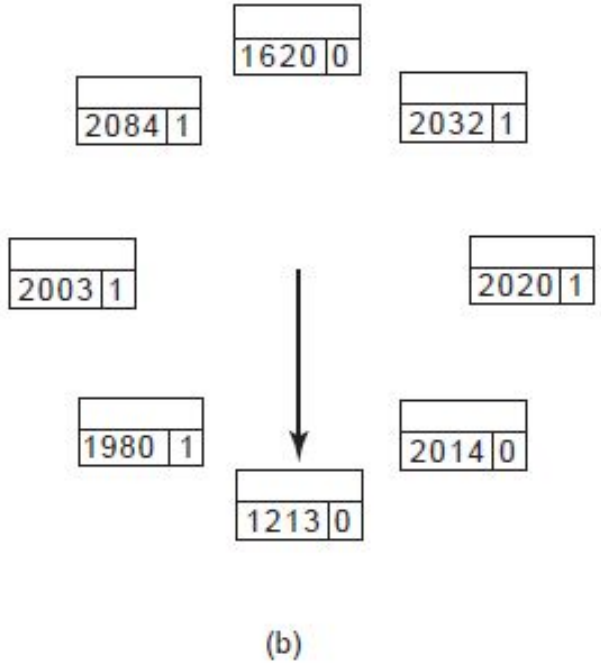
This leads to algorithms like WSClock (Working Set + Clock).

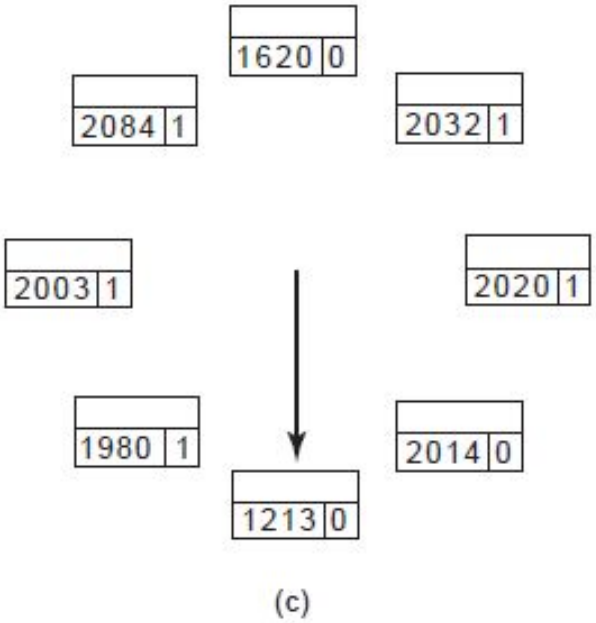
Example

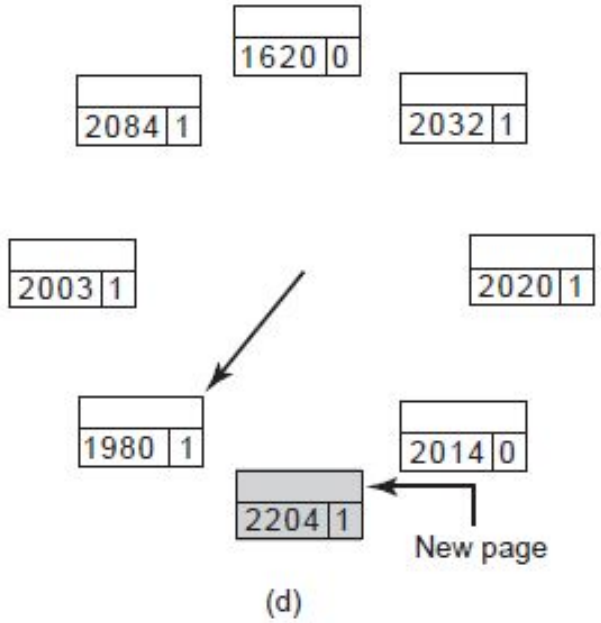
- $\Delta = 5$ time units
- Current time = 10
- Pages in memory:
  - Page A: last used at time 8 → in working set ($8 \geq 10–5 = 5$)
  - Page B: last used at time 3 → not in working set ($3 < 5$)

→ Evict Page B.


1620 0
2084 1
2032 1
2003 1
2020 1
1980 1
2014 1
1213 0
Time of last use
R bit
(a)


1620 0
2084 1
2032 1
2003 1
2020 1
1980 1
2014 0
1213 0
(b)


1620 0
2084 1
2032 1
2003 1
2020 1
1980 1
2014 0
1213 0
(c)


1620 0
2084 1
2032 1
2003 1
2020 1
1980 1
2014 0
2204 1
New page
(d)

Review of Page Replacement Algorithms

Algorithm	Comment
Optimal	Not implementable, but useful as a benchmark
NRU (Not Recently Used)	Very crude
FIFO (First-In, First-Out)	Might throw out important pages
Second chance	Big improvement over FIFO
Clock	Realistic
LRU (Least Recently Used)	Excellent, but difficult to implement exactly
NFU (Not Frequently Used)	Fairly crude approximation to LRU
Aging	Efficient algorithm that approximates LRU well
Working set	Somewhat expensive to implement
WSClock	Good efficient algorithm

- the two best algorithms are aging and WSClock.
  - based on LRU and the working set, respectively.
- Both give good paging performance and can be implemented efficiently.

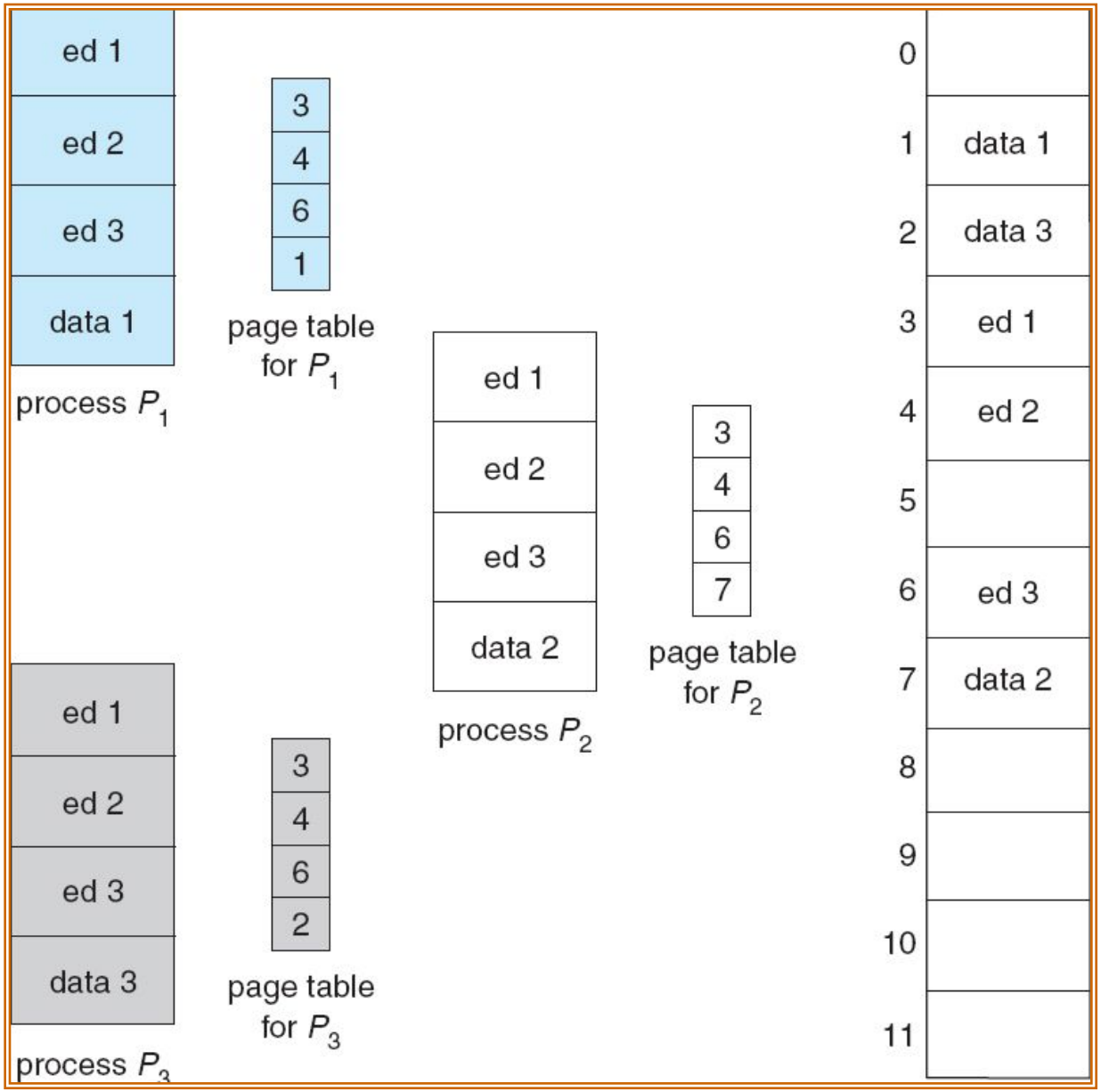
Some Definitions

- Demand Paging:
  - Pages are only loaded when accessed
  - When process begins, all pages marked INVALID
- Prepaging:
  - loading the pages before letting processes run
- A program causing page faults every few instructions is said to be thrashing

Shared Pages

- **Processes can share pages**
  - **Entries in page tables point to the same physical page frame**
- **Example**
  - **Multiple text editors used at the same time**
  - One copy of read-only (reentrant) program text shared among the processes (i.e., text editors).
  - Each process keeps a separate copy data

Shared Pages Example



Operating System's Involvement with Paging

Four times when OS involved with paging

- Process creation
  - Determine program size
  - Create page table
- During process execution
  - Reset the MMU for new process
  - Flush the TLB (or reload it from saved state)
- Page fault time
  - Determine virtual address causing fault
  - Swap target page out, needed page in
- Process termination time
  - Release page table
  - Return pages to the free pool

Two Views of A Virtual Address Space

- The virtual memory discussed so far is one-dimensional
  - the virtual addresses go from 0 to some maximum address, one address after another
- Users prefer to view memory as a collection of variable-sized segments, with no necessary ordering among segments

Two Views of A Virtual Address Space

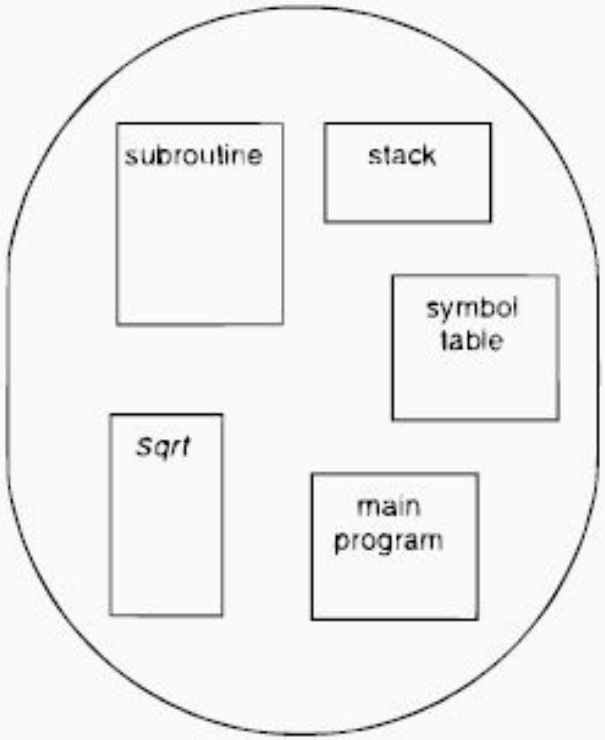
One-dimensional

a linear array of bytes

```
max +------------------+
    |      Stack       | Stack segment
    +--------+---------+
    |        |         |
    |        v         |
    |                  |
    |        ^         |
    |        |         |
    +--------+---------+
    | Dynamic storage  | Heap
    |(from new, malloc)|
    +------------------+
    | Static variables |
    | (uninitialized,  | BSS  segment
    |  initialized)    | Data segment
    +------------------+
    |      Code        | Text segment
0   +------------------+
```

Two-dimensional

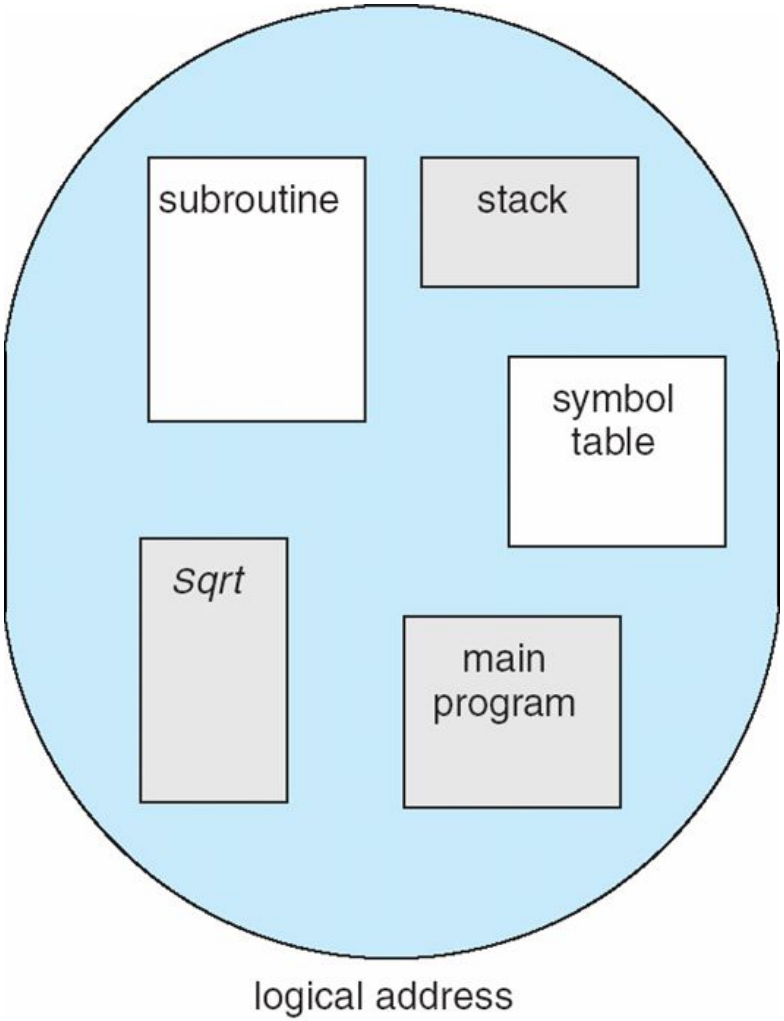
a collection of variable-sized segments



Segmentation

- Memory-management scheme that supports user view of memory
- A program is a collection of segments
  - A segment is a **logical unit** such as:
    - main program
    - procedure
    - function
    - method
    - object
    - local variables, global variables
    - common block
    - stack
    - symbol table
    - arrays

User's View of a Program

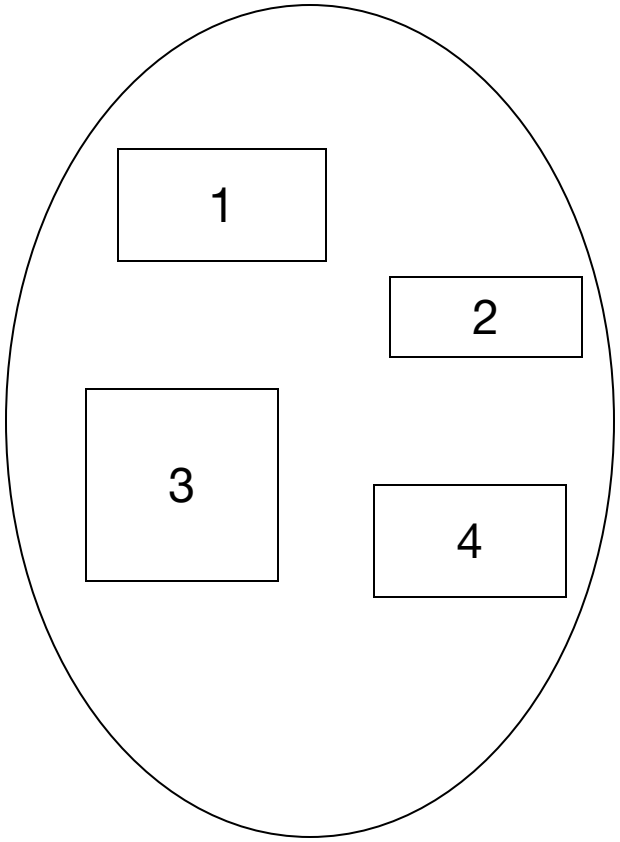


User's View of a Program

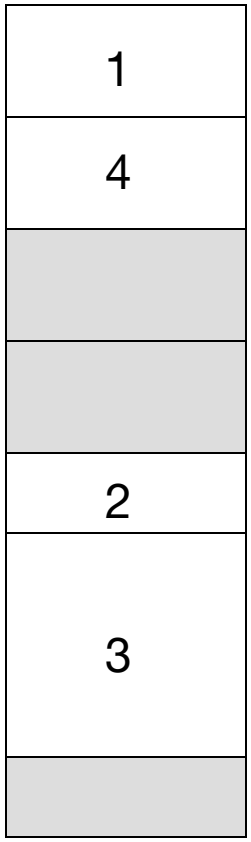
```
int global_var = 10;          // → goes in "Data" segment

void main() {                 // → code in "Code" segment
   int x = 5;                 // → stored in "Stack" segment
   int *p = malloc(100);      // → allocated in "Heap"
segment
}
```

Logical View of Segmentation



user space

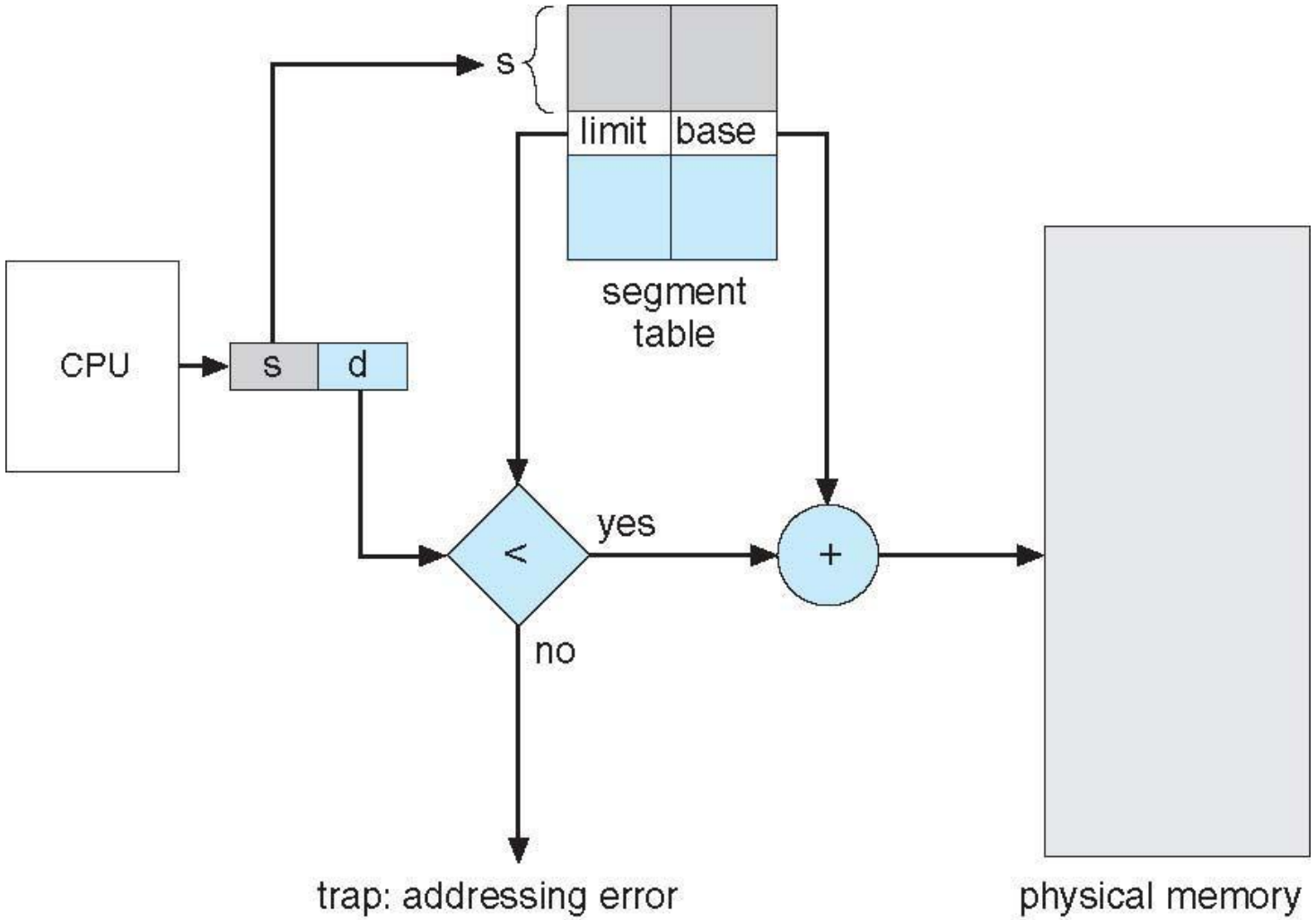


physical memory space

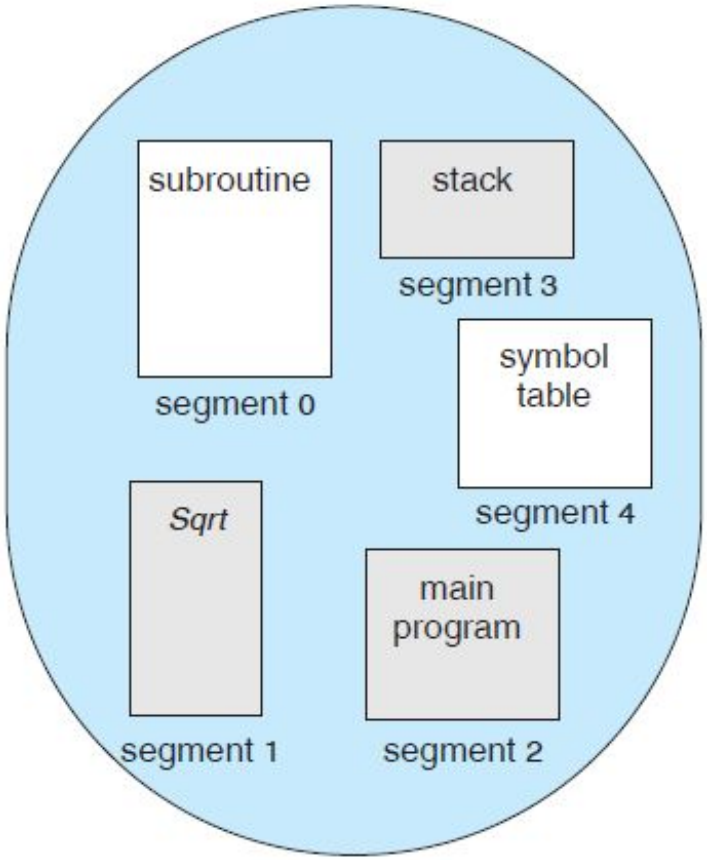
Segmentation Architecture

- Logical address consists of a two tuple:
  <segment-number, offset>,
- **Segment table** – maps two-dimensional logical addresses into one-dimensional physical addresses;
  - each table entry has:
    - **base** – contains the starting physical address where the segments reside in memory
    - **limit** – specifies the length of the segment

Segmentation Hardware

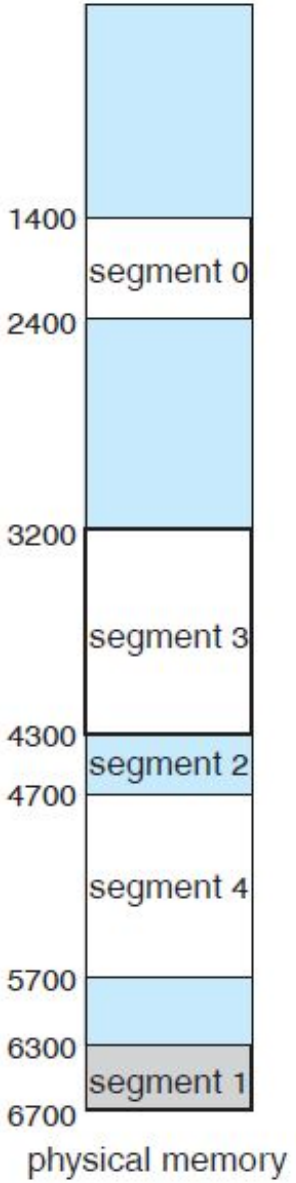


Example of Segmentation



	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table



$(2, 53) \Rightarrow 4300 + 53 = 4353$

$(3, 852) \Rightarrow 3200 + 852 = 4052$

$(0, 1222) \Rightarrow$ Trap!

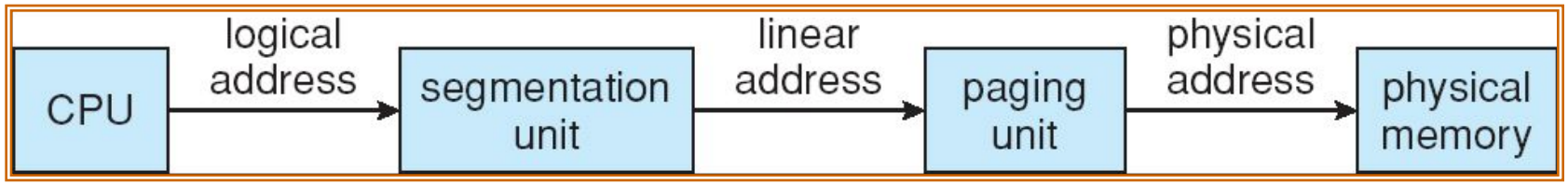
Segmentation

Consideration	Paging	Segmentation
Need the programmer be aware that this technique is being used?	No	Yes
How many linear address spaces are there?	1	Many
Can the total address space exceed the size of physical memory?	Yes	Yes
Can procedures and data be distinguished and separately protected?	No	Yes
Can tables whose size fluctuates be accommodated easily?	No	Yes
Is sharing of procedures between users facilitated?	No	Yes
Why was this technique invented?	To get a large linear address space without having to buy more physical memory	To allow programs and data to be broken up into logically independent address spaces and to aid sharing and protection

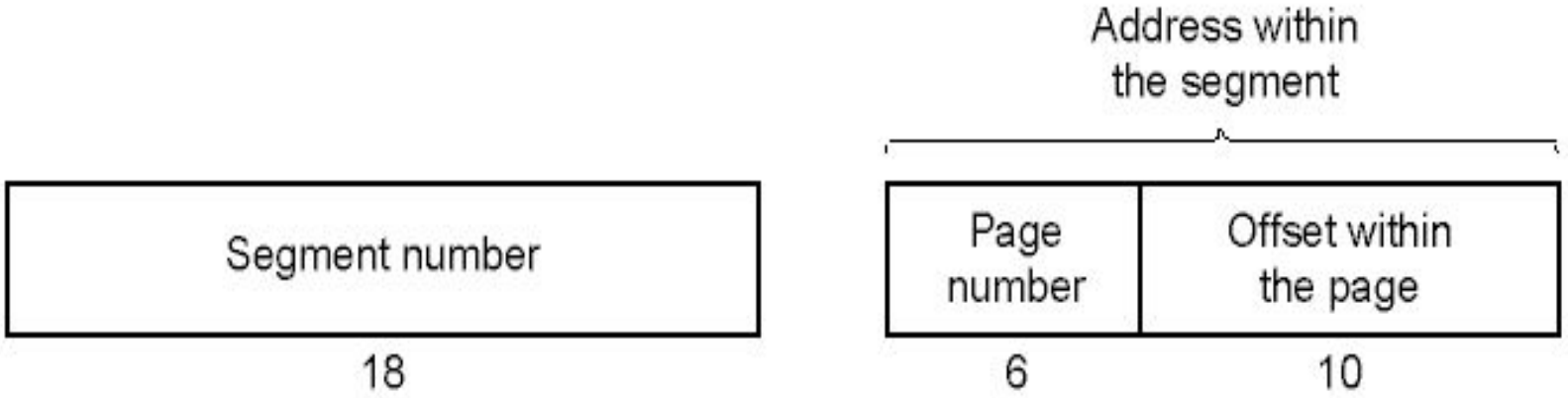
Comparison of paging and segmentation

Segmentation with Paging

- If the segments are large
  - it may be inconvenient, or even impossible, to keep them in main memory in their entirety.
- This leads to the idea of paging them
  - only those pages that are actually needed have to be around

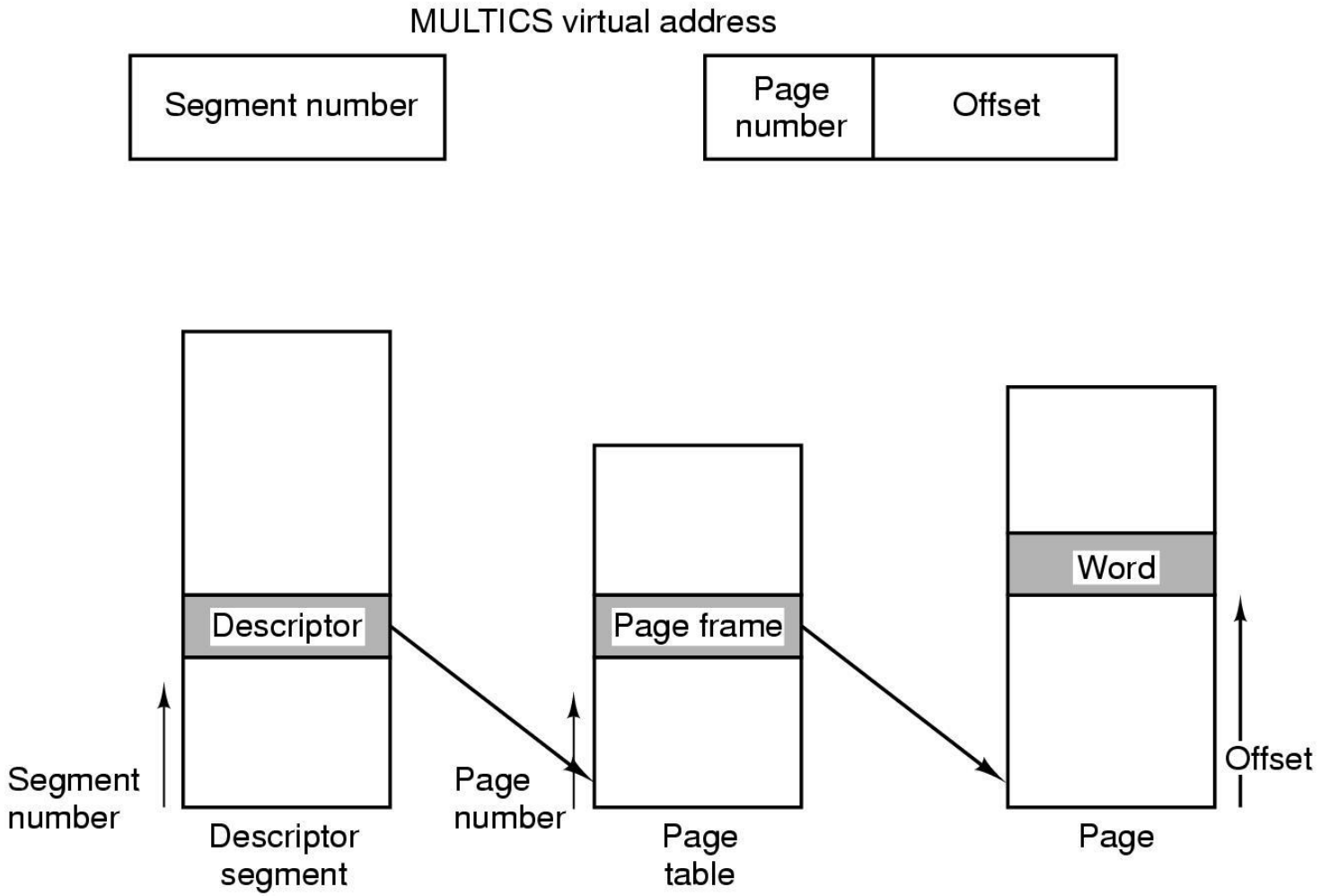


Segmentation with Paging: MULTICS



A 34-bit MULTICS virtual address

Segmentation with Paging: MULTICS (3)



Conversion of a 2-part MULTICS address into a main memory address